Exmentify

AI-Powered Exam Preparation Companion — Project Report

Submitted for: IBM SkillsBuild BharatCares / AICTE Internship — Gen AI and Cloud Computing

Developed by: Sheema Shariq

Course: BSc (hons) Computer Science

Institution: Jamia Millia Islamia

Live Application

Primary URL (HTTPS via CloudFront):
https://d1vcdm3hqyivum.cloudfront.net/

Backend URL (HTTP via AWS Elastic Beanstalk):
http://sheema-exmentify-app.ap-south-1.elasticbeanstalk.com/

Note: The application is hosted on AWS Elastic Beanstalk. A CloudFront distribution sits in front of it to serve the app over HTTPS. Both URLs are live and point to the same application.

To see live AI-generated responses, click the GATE - Computer Science (CS) card on the homepage. The other exam cards (other GATE branches, UGC NET, CAT, GRE, etc.) are shown as "Coming Soon" to demonstrate that the application's architecture is exam-agnostic, but only GATE CS has topics, PYQs, and AI generation fully wired up in this submission.

1. Application Overview and Tech Stack

Exmentify is an AI-powered exam preparation companion built for students preparing for competitive and postgraduate entrance exams in India — spanning GATE (all branches), UGC NET, CSIR NET, CAT, and GRE, with campus placement and government-exam tracks also represented. The core idea: a student picks an exam, a subject, and a topic, and receives AI-generated concept explanations, revision notes, practice questions, and previous-year-question (PYQ) solutions — streamed token-by-token rather than delivered as a single block, so it reads like a live tutor rather than a static answer dump.

Beyond the four core AI modes, Exmentify includes a Weekly Contest mode (student-selected topics, timed, with personal — never comparative — feedback), an open-ended doubt-resolution chat, AI-generated flashcards, a 'Diagnose My Mistake' tool that explains the misconception behind a specific wrong answer, an AI study-plan generator tied to a real exam date, structured cheat sheets, and a full study-tracking layer (streaks, daily goals, bookmarks, and achievement badges) — all built without a backend database, using the browser's local storage for every piece of personalization.

Technology Stack

Layer	Choice	Notes
Backend	Python 3.11 + FastAPI	Chosen for native StreamingResponse support, required for token-by-token output
LLM	Google Gemini API (google-genai SDK)	gemini-flash-lite-latest — free-tier accessible, high daily quota
Frontend	Vanilla HTML / CSS / JavaScript	No framework — keeps the Docker image small, no build step
State / storage	Browser localStorage	No database — all personalization (streaks, bookmarks, theme, contest history) lives client-side
Diagrams	Mermaid.js (CDN)	Renders inline flowcharts/trees when Gemini determines a visual aids an explanation
PDF export	jsPDF (CDN)	Client-side export of notes, cheat sheets, PYQ sets, and study plans
Containerization	Docker	Single Dockerfile serves both the static frontend and the API from one image
Deployment	AWS Elastic Beanstalk + CloudFront	Elastic Beanstalk runs the container; CloudFront terminates HTTPS in front of it
Secrets	Environment variables (.env locally / EB environment properties in production)	GEMINI_API_KEY is never committed to source control or baked into the Docker image

2. Prompting Strategy and Frameworks Used

Every AI-generated feature in Exmentify follows the same underlying pattern: a fixed system prompt defines the assistant's role and output constraints, while a dynamically built user prompt supplies the specific subject, topic, and task context. All prompts share a common 'plain text' constraint, since the frontend renders responses as plain text rather than parsed Markdown — an early version let Gemini's default Markdown formatting (##, **, $$) leak into the UI as literal symbols, which was fixed by explicitly instructing the model not to use Markdown, LaTeX, or code fences anywhere except a single permitted case (see Diagrams, below).

The very first Prompt I gave to build the App.

Imagine you are a full stack developer. I want you to create a web app called Exmentify ,an AI-powered exam preparation companion for students. Core concept: A student picks an exam (e.g. GATE - Computer Science) and a subject (e.g. Data Structures & Algorithms), then gets AI-generated concept explanations, revision notes, practice questions, and previous-year-question (PYQ) solutions — streamed progressively like ChatGPT, not dumped all at once. There's also a weekly contest mode that gives personalized, time-based feedback instead of leaderboard rankings, so students never feel compared to others. Tech stack:

- Backend: Python 3.11 with FastAPI (use StreamingResponse for all AI-generated content)
- LLM: Anthropic Claude API (claude-sonnet-4-6) using the official Python SDK, called only from the backend — API key must live in environment variables, never in frontend code or git
- Frontend: Vanilla HTML/CSS/JS, no framework, responsive for mobile and desktop
- State/storage: No database — use localStorage in the browser to store user progress, contest history, and theme preference
- Seed data: Bundle static JSON files in the app for the exam catalog, subjects/topics, and a set of ~20-30 PYQs
- Containerize with Docker (single Dockerfile serving both frontend and backend) and deploy to AWS App Runner with a public HTTPS URL Key features to build:

1. Exam-select screen showing GATE CS as fully active, and at least 2-3 other exams (e.g. UGC NET, CAT, UPSC CSE) shown as disabled "Coming Soon" cards — this should prove the app is exam-agnostic even though only GATE CS has real content.
2. Subject/topic browser for GATE CS, starting with DSA (Arrays, Trees, Graphs, DP) with a weightage tag (low/medium/high) per topic.
3. Four AI-powered modes per topic, each streaming text token-by-token: Explain a concept, Generate revision notes, Generate practice questions, Solve a specific PYQ step-by-step.
4. A weekly contest mode: timed questions pulled from the PYQ bank, tracks time-per-question and correctness in localStorage, and afterward calls the AI to generate personalized, non-comparative feedback (what to revise, not "you ranked #12").
5. A reschedule control for the contest that only allows picking a new date within 7 days of the original — reject and explain if outside that window.
6. Dark/light theme toggle that persists across reloads. Backend API endpoints (all POST endpoints stream via SSE):

- GET /api/health
- GET /api/exams
- GET /api/subjects/{exam_id}
- GET /api/pyqs/{exam_id}/{topic_id} (don't leak the correct answer in list view)
- POST /api/explain, POST /api/notes, POST /api/practice, POST /api/pyq-solution — all take {exam_id, subject_id, topic_id, mode, language, extra} and stream back generated content
- POST /api/contest-feedback — takes contest results and streams back personalized recommendations Edge cases you must handle gracefully (no crashes, no silent wrong data):
- Invalid exam/topic combination → return 400 with a clear message.

- LLM API failure or timeout mid-stream → show a "couldn't generate this, retry" UI state, never an infinite spinner.
- Empty PYQ/practice bank for a topic → return 200 with an empty array and a friendly "not seeded yet" message.
- Reschedule date outside the 7-day window → reject with explanation.
- Verify the API key never appears in frontend JS, network responses, or the Docker image

Shared plain-text constraint

```
Write in plain text only — no Markdown syntax at all (no #, ##, **, $$,
backticks, or bullet dashes). Use plain paragraph breaks and, if needed,
simple numbered lines like '1.' for structure. Write math inline in
plain words or simple notation (e.g. 'O(n log n)', 'i * 20 + j')
instead of LaTeX.
```

Sample prompt — Explain mode (with conditional diagram support)

System prompt:

```
You are a patient, encouraging exam-prep tutor for Indian competitive
exams. Explain the concept clearly with a short intuition, a worked
example, and a one-line summary. Keep it focused and exam-relevant.
If a simple diagram would meaningfully aid understanding (e.g. a tree,
a graph, a state machine, a pipeline), include exactly ONE Mermaid
diagram, wrapped in a single fenced block. Place it right after the
intuition section. If a diagram would not genuinely help, omit it
entirely — do not force one.
```

This is a deliberate exception carved into an otherwise strict no-Markdown rule: the frontend detects a single fenced mermaid block in the completed response, extracts it, and renders it as a real inline diagram using Mermaid.js, while the remaining text displays normally. The model decides per-response whether a diagram is warranted, rather than one being forced on every explanation.

Sample prompt — Practice mode (difficulty-tuned)

The Practice mode's system prompt requires strict JSON output (question, four options, correct index, explanation) so the frontend can render it as interactive quiz cards rather than raw text. The user prompt is built dynamically per request:

```
"Subject: {subject}\nTopic: {topic}\nLanguage: {language}\nGenerate {count} {difficulty}-
difficulty practice questions on this topic."
```

The {difficulty} slot is filled from a frontend dropdown (Easy / Medium / Hard), giving the same topic meaningfully different question sets on request — a direct, visible demonstration of prompt parameterization rather than a single static prompt per feature.

Sample prompt — Diagnose My Mistake

This feature intentionally asks the model to reason about the student's error, not just restate the correct answer:

```
Diagnose the likely misconception behind this specific wrong answer —
not just why the correct answer is correct, but what mental model or
mistake probably led the student here. Be specific and kind, not generic.
```

Token budget tuning per mode

A single global token limit was tried first and rejected — it either truncated longer content (Explain, Practice) or wasted budget on short content (Contest Feedback). Each mode now has an independently tuned ceiling:

Mode	Max tokens	Reasoning
Explain	2,500	Intuition + worked example + summary, plus an optional diagram
Revision Notes	1,200	Deliberately short and skimmable, not a full essay
Practice	4,000	Up to 5 full questions, each with 4 options and an explanation
PYQ Solutions	2,500	Multi-step reasoning needs room to breathe
Ask a Doubt	900	A direct, concise answer to one specific question
Contest Feedback	800	A short, personal paragraph — not a report
Study Plan	3,000	A full day-by-day or week-by-week schedule
Cheat Sheet	3,000	4–6 structured points across every topic in a subject

3. Phase-by-Phase Development Summary

Phase 1 — Architecture and Scaffolding

Defined the FastAPI backend structure, the static-file-serving frontend, the seed-data schema (exams → subjects → topics → PYQs), and the Dockerfile. Established the principle that AI-generated content (Explain, Notes, Practice, PYQ Solutions) would always be generated live rather than pre-written, while only the PYQ bank itself needed authored seed content.

Phase 2 — LLM Provider Integration and Migration

The application was originally scaffolded against the Anthropic Claude API. Mid-project, the LLM provider was switched to Google Gemini to work within a free-tier API key rather than a paid one — a real cost-and-access constraint that shaped the rest of the build. This meant replacing the Anthropic SDK entirely with google-genai, rewriting the streaming logic around Gemini's generate_content_stream interface, and re-tuning every prompt and token budget for the new model family.

Phase 3 — Core Feature Build

Implemented the four core AI modes (Explain, Revision Notes, Practice, PYQ Solutions), the SSE streaming pipeline end-to-end, the Weekly Contest mode with a timer and scoring, and the initial DSA/OS-only PYQ seed bank.

Phase 4 — Debugging and Reliability Hardening

A significant portion of development time went into diagnosing and fixing real-world reliability issues rather than adding features — detailed fully in Section 5. This phase produced the automatic .env loading logic, the retry-with-backoff mechanism for transient server errors, and the diagnostic logging that made every subsequent bug visible instead of silent.

Phase 5 — Content Expansion

Expanded topic coverage from 2 subjects (DSA, OS) to the full official GATE CS syllabus — 10 subjects, 45 topics — and grew the PYQ bank from 24 to 79 seeded questions, ensuring every topic has at least one real previous-year question for the PYQ Solutions and Contest features. The exam catalog was also expanded to include other GATE branches (DA, EE, CE, ME, ECE) and other exam types spanning postgraduate, PhD, and job-track paths (UGC NET, CSIR NET, CAT, GRE, Campus Placement, Bank & SSC), to genuinely reflect the app's exam-agnostic architecture.

Phase 6 — UX and Engagement Features

Added PDF export (via jsPDF, with a safe-save fallback for restricted browser contexts), a 4-theme system (Study Lamp, Daylight, Midnight, Sepia), a study streak tracker, a daily-goal progress ring, topic bookmarking, and an achievement badge system — all persisted through localStorage with no backend changes required.

Phase 7 — Advanced AI Features

Extended the AI surface area beyond the four core modes: an open-ended 'Ask a Doubt' chat scoped to the current topic with conversation history, AI-generated flashcards with a flip-card UI, the 'Diagnose My Mistake' misconception tool, an AI study-plan generator tied to a real exam date, Mermaid-diagram support inside Explain responses, and a redesign of the Cheat Sheet feature from freeform text into structured, parsed JSON rendered as color-coded topic cards.

Phase 8 — Cloud Deployment

Deployed the Dockerized application to AWS Elastic Beanstalk. Since Elastic Beanstalk's default *.elasticbeanstalk.com domain cannot be issued a valid SSL certificate without a custom domain, a CloudFront distribution was placed in front of it to provide the public HTTPS endpoint required by the assignment brief, while Elastic Beanstalk continues to serve as the underlying compute layer.

4. Application Architecture

Exmentify is intentionally a single-container, database-free architecture. One Docker image contains both the FastAPI backend and the static frontend files; FastAPI serves the frontend directly via StaticFiles, so there is exactly one running process and one deployable unit.

Request flow

- Browser loads the static frontend (HTML/CSS/JS) directly from the FastAPI server.
- User interactions (Generate, Ask a Doubt, Start Contest, etc.) trigger fetch() calls to /api/* endpoints.
- Catalog endpoints (/api/exams, /api/subjects, /api/pyqs) return static JSON read from seed data files at startup.
- AI-generation endpoints (/api/explain, /api/notes, /api/practice, /api/pyq-solution, /api/ask-doubt, /api/flashcards, /api/diagnose-mistake, /api/study-plan, /api/cheat-sheet, /api/contest-feedback) each build a prompt, call the Gemini API via streaming, and relay chunks to the browser as Server-Sent Events.
- The frontend parses the SSE stream manually via fetch() + ReadableStream (since EventSource does not support POST requests), rendering text progressively as it arrives.
- All personalization (bookmarks, streaks, theme, contest history, daily goals, badges) is computed and stored entirely in the browser's localStorage — no user data is persisted server-side.

Cloud deployment architecture

- AWS Elastic Beanstalk hosts the Docker container as a single-instance environment, exposing the app over HTTP on its default domain.
- An AWS CloudFront distribution sits in front of the Elastic Beanstalk origin, terminating HTTPS using AWS's managed certificate on the CloudFront domain and forwarding requests to the Elastic Beanstalk origin over HTTP.
- GEMINI_API_KEY is injected as an Elastic Beanstalk environment property at deploy time — never committed to source control, never baked into the Docker image.

5. Challenges Encountered and How They Were Resolved

Challenge	Resolution
Windows dev environment friction: uvicorn.exe blocked by Windows Smart App Control; PowerShell script execution disabled by default; python vs python3 command mismatch.	Standardized on 'python -m uvicorn' (routes through the trusted python.exe rather than the flagged wrapper executable); documented Set-ExecutionPolicy -Scope CurrentUser RemoteSigned as a one-time fix; used 'python' consistently per Windows convention.
LLM provider needed to change mid-project from Anthropic Claude to Google Gemini for free-tier accessibility.	Fully replaced the anthropic SDK with google-genai, rewrote the streaming call around generate_content_stream, and re-validated every prompt and token budget against the new model.
GEMINI_API_KEY set in a local .env file was not being read by the app when run outside Docker — 'AI service is not configured' errors despite the key being present.	Added python-dotenv with a multi-path search (current working directory walked upward, project root, and the backend folder itself) so the key loads correctly regardless of which directory the terminal is in when uvicorn is launched.
Google retired the originally integrated model (gemini-2.5-flash) for new API keys mid-development, returning a 404 'no longer available' error with no advance warning.	Switched to gemini-flash-latest — Google's auto-updating alias that always resolves to the current-generation flash model — so a future model retirement will not silently break the app again.
The free-tier daily quota for the newest Gemini model (20 requests/day) was exhausted during active development and testing, returning 429 RESOURCE_EXHAUSTED errors.	Switched to gemini-flash-lite-latest, a lighter model tier with a substantially higher free daily quota, fully sufficient for both development and a live evaluation session.
Gemini occasionally returned transient 503 'model is currently experiencing high demand' errors, particularly on the free tier during peak hours.	Implemented automatic retry logic (up to 3 attempts with short backoff) specifically for transient errors, while genuine errors (invalid key, bad request) still fail immediately without wasting retries.
Errors were being silently swallowed server-side, giving no visibility into why a request failed — made debugging every subsequent issue slower.	Added diagnostic logging (with the API key redacted) on every failure, plus a startup log confirming whether the key was found at all, so the real cause is always visible in the terminal.
Gemini's default Markdown formatting (##, **, $$) rendered as literal symbols in the UI, since the frontend displays plain text, not parsed Markdown.	Added an explicit plain-text constraint to every system prompt, with one deliberate, narrow exception for a single Mermaid diagram block in Explain mode, which is detected and rendered separately.
Button elements rendered with invisible black text against the dark theme — browsers do not automatically inherit text color into <button> elements the way they do for other elements.	Added a global 'button { color: inherit; }' reset, fixing every current and future button in the app at once rather than patching each instance individually.
Switching between workspace tabs (Flashcards, Diagnose Mistake, Ask a Doubt) showed multiple panels stacked on top of each other instead of only the active one.	Traced to custom 'display' CSS rules silently overriding the browser's built-in [hidden] attribute behavior; fixed with a single global '[hidden] { display: none !important; }' rule placed above all other styles.
AWS Elastic Beanstalk's default domain cannot be issued a valid HTTPS certificate, since AWS controls that domain and will not certificate it for an individual environment — yet the assignment brief requires a public HTTPS URL.	Placed an AWS CloudFront distribution in front of the Elastic Beanstalk origin, using CloudFront's own managed HTTPS certificate to serve the app securely without requiring the purchase of a custom domain.

6. Key Learnings and Reflection

Free-tier LLM APIs are a real engineering constraint, not just a cost line item

Model deprecation and daily quota limits were not hypothetical risks — both were hit during active development. Building around a free-tier API meant treating quota exhaustion and model retirement as expected failure modes to design for (auto-updating model aliases, retry logic, clear error surfacing), not edge cases to ignore.

Defensive engineering compounds

Each individual fix — dotenv path searching, retry logic, diagnostic logging, the plain-text prompt constraint — was small on its own. Together, they turned a fragile integration that failed silently into one that fails loudly, recovers automatically where possible, and is fast to diagnose when it can't. Most of the debugging time in this project went into building the tools to see what was wrong, not into the eventual one-line fix once it was visible.

Prompt engineering is iterative, not a first draft

Nearly every system prompt in this project was rewritten after seeing real output: the plain-text constraint only emerged after Markdown symbols showed up raw in the UI; per-mode token budgets only emerged after Explain responses were seen cutting off mid-sentence. Prompt engineering here meant treating the model's actual output as the specification to correct against, not something to get right purely by reasoning about the prompt in isolation.

Platform-specific issues need platform-specific fixes

Windows-specific quirks (Smart App Control blocking uvicorn.exe, PowerShell's execution policy, VS Code's automatic — and sometimes wrong — virtual environment detection) were not covered by general Python/FastAPI documentation, which tends to assume a Unix-like environment. Diagnosing these required treating the actual terminal output as ground truth rather than assuming the standard instructions would simply work.

Cloud deployment has non-obvious constraints that must be discovered, not assumed

The assumption that any AWS compute service would trivially provide HTTPS turned out to be wrong for Elastic Beanstalk specifically — a constraint only surfaced by actually attempting the deployment and hitting the certificate limitation, not something obvious

from the service's marketing description. Working around it (CloudFront in front of Elastic Beanstalk) reflects a genuinely common real-world AWS architecture pattern, not a one-off workaround.

Reflection

Building Exmentify end-to-end — from initial scaffolding through a full provider migration, a long list of real production bugs, a significant feature expansion, and finally a cloud deployment with its own unexpected constraint — mirrored the actual shape of real software development far more than a clean, uninterrupted build would have. The 'vibe coding' methodology of rapid AI-assisted iteration did not remove the need for testing, verification, and architectural judgment; if anything, it made those skills more central, since the speed of generating code meant the bottleneck shifted to verifying that code actually worked as intended, in the actual environment it needed to run in.